

# QA Process

APPN Aerial SOP — Locked revision 1.0

## APPN – Aerial Data QC Protocols

### Note

This document describes procedures for measuring the uncertainty of drone flights. Initially this document focuses on the hyperspectral drones, but the procedure will expand in the future. It will also form the basis of a standard QC procedure for future flights.

### Note

The QC procedures in this document are tightly coupled to the APPN flight designs — the panels, ground control points, and reflectance targets that the QC steps rely on are deployed *during* the flight itself. Before performing any QC, make sure you understand which targets were placed in the field by reading the relevant flight protocol:

- Standard Flight — routine APPN data-collection flight; defines the baseline panel and GCP layout used for ELM and positional QC.
- Validation Flight — extended flight that adds additional validation panels, GCPs, and (in future) LiDAR calibration targets used by the spectral, positional, and LiDAR QC sections below.

## Document Structure

The conventions in General QC Conventions apply to **all** QC processes below. Read that section first, then jump to the specific QC type you need:

- General QC Conventions
- File Format — GeoJSON vs Shapefile
- Naming Conventions
- File Storage
- Positional QC
- Spectral QC
- Creating Geospatial Vector Data — GOBI & CALViS (QGIS)
- Extracting Pixels into a Table
- LiDAR QC

---

## General QC Conventions

These file-format, naming, and storage rules apply to every QC process (positional, spectral, LiDAR) described later in this document.

## File Format — GeoJSON vs Shapefile

### Important

**GeoJSON (.geojson) is the preferred format** for all QC vector data, though shapefiles (.shp) are also supported by the QA and plot extraction code [https://github.com/ArdenB/APPN\\_GenricFileStorage](https://github.com/ArdenB/APPN_GenricFileStorage).

GeoJSON is preferred because:

- It is a **single self-contained file**, rather than the 4-6 sidecar files (.shp, .shx, .dbf, .prj, .cpg, ...) required by the shapefile format. This makes it easier to copy, share, and store without losing pieces.
- It is **plain-text JSON**, so it is human-readable, diff-friendly, and works cleanly with Git version control.
- It is an **open, web-native standard** (RFC 7946) supported by virtually all modern GIS tools, web mappers, and Python/R libraries.
- CRS handling is explicit: the default GeoJSON specification (RFC 7946) only supports WGS84 (EPSG:4326), but all major GIS software (QGIS, ArcGIS, GDAL, geopandas, etc.) allow you to specify and read a different CRS when writing/reading the file. For APPN QC work we **keep the file in the same projected CRS as the source orthomosaic** (e.g. GDA2020 / MGA zone XX) rather than reprojecting to WGS84.
- It has **no field-name length limit** (shapefile DBF caps at 10 characters) and no 2 GB file-size cap.

If you already have shapefiles, you do **not** need to re-create them — the QA pipeline will read either format. New files should be saved as .geojson.

## Naming Conventions

| Name                                  | Overview                                                                                                                 |
|---------------------------------------|--------------------------------------------------------------------------------------------------------------------------|
| QC_ELM_Panels.geojson                 | Polygons of the reflectance panels used in the ELM during GRYFN Processing.                                              |
| QC_VAL_Grfyn_Panels.geojson           | When a second set of GRYFN panels is placed in the field. Replace <i>ELM</i> with <i>VAL</i> for validation.             |
| QC_VAL_{PanelName}_Panels.geojson     | Any future validation panels or tests of other panels. Replace {PanelName} with the name or unique identifier.           |
| QC_GCP_groundtruth_points.geojson     | <b>Reference</b> GCP locations measured independently in the field (e.g. Aeropoint, Trimble RTK). Points-only file; one  |
| QC_GCP_points.geojson                 | <b>Observed</b> GCP locations as digitised from the drone orthomosaic in QGIS. Points-only file; GCP_name must match the |
| QC_LIDAR_{TargetName}_Surface.geojson | Name for any future LiDAR calibration surfaces.                                                                          |

### Tip

Any additional information (e.g. date) can be added to the end of the file name with an underscore — e.g. QC\_ELM\_Panels\_20260302.geojson. This table will be updated if we source panels other than GRYFN. Shapefile equivalents (e.g. QC\_ELM\_Panels.shp) are also accepted by the QA code.

## File Storage

Any files created in the quality control steps are to be stored in a newly created QC\_data folder inside T1\_proc, following the APPN folder structure.

Formal path:

```
./{Node}/  
{YYYY_ProjectDesc[_I|E][_Researcher][_org]}/  
{YYYYSiteName[_F|C]}/  
{SensorPlatform}/{YYYYMMDD}/run_XX/T1_proc/QC_data/
```

Example:

```
./USYD_Narrabri/2025_SIFCal/2025IAWatson_F/CALVIS/20250825/run_00/T1_proc/QC_data/
```

These vector files (GeoJSON preferred, shapefile accepted) are meant to be created after the GPRO has been completed. The APPN storage repo has been updated to include this folder: [https://github.com/ArdenB/APPN\\_GenricFileStorage](https://github.com/ArdenB/APPN_GenricFileStorage)

## Positional QC

### Warning

This section is still a work in progress.

### Important

Positional QC **must be performed immediately after processing is complete** (e.g. straight after GPRO creation). Positional errors are often the first indication of a larger problem with the flight or the processing pipeline, and almost always require the data to be reprocessed before they can be resolved. Catching them early avoids wasted downstream work.

APPN Positional QC is performed in three stages:

1. **Field Data Collection**
2. **Observed point capture**
3. **Accuracy reporting**

The descriptions below show the procedure for the CALViS using GCPs. The process for the GOBI is the same, but there is only the VNIR orthomosaic. The same workflow can be used to produce shapefiles if preferred — just choose *ESRI Shapefile* in place of *GeoJSON* in the format dropdown.

For the CALViS, in the products folder of the completed GPRO you will find the **.tif** files:

- X\_VNIR\_Orthomosaic.rgb
- X\_SWIR\_Orthomosaic.rgb

These are the RGB bands from the VNIR and SWIR files respectively. They can be easier to use than the full **.bin** files, though the procedure is the same.

### Field Data Collection

Independent GCPs are placed in the field during data capture (see Standard Flight and Validation Flight). These GCPs are independent of any points used during processing so they provide an unbiased reference ("ground truth") to compare the drone-derived positions against.

The **raw** data exported from the GCP hardware (Aeropoint CSVs, Trimble job files, etc.) should be saved to the **T0\_raw/Vault** folder so the original measurements are preserved.

Formal path:

```
./{Node}/  
{YYYY_ProjectDesc[_I|E][_Researcher][_org]}/  
{YYYYSiteName[_F|C]}/  
{SensorPlatform}/{YYYYMMDD}/run_XX/T0_raw/Vault/
```

Example:

```
./USYD_Narrabri/2025_SIFCal/2025IAWatson/CALVIS/20250825/run_00/T0_raw/Vault/
```

The raw data must then be converted into the APPN standard reference format: a GeoJSON with one point per GCP and the fields ID, X, Y, Z, plus an explicit CRS. The exact conversion process depends on the GCP hardware (Aeropoint, Trimble, etc.).

### Important

**TODO:** Document the conversion process for each supported GCP type (Aeropoint, Trimble, ...).

Common issues to watch for during conversion:

- Mismatched CRS between GCP collection and data processing.

- Incorrect height datum — e.g. GDA2020 ellipsoidal heights vs the Australian Height Datum (AHD). See Geoscience Australia's AUSGeoid2020 conversion tool for converting between the two.
- Inconsistent ID names or duplicate points — fix or remove before saving.

Once the issues are resolved, save the cleaned reference points as `QC_GCP_groundtruth_points.geojson` in the `T1_proc/QC_data/` folder (see Naming Conventions and File Storage).

Formal path:

```
./{Node}/
{YYYY_ProjectDesc[_I|E][_Researcher][_org]}/
{YYYYSiteName[_F|C]}/
{SensorPlatform}/{YYYYMMDD}/run_XX/T1_proc/QC_data/QC_GCP_groundtruth_points.geojson
```

Example:

```
./USYD_Narrabri/2025_SIFCal/2025IAWatson/CALVIS/20250825/run_00/T1_proc/QC_data/QC_GCP_groundtruth_points.
```

## Observed point capture

Manually digitise a matched set of points from the drone orthomosaic in QGIS and save them as `QC_GCP_points.geojson` (see Naming Conventions). The `GCP_name` column **must** match the ID values used in `QC_GCP_groundtruth_points.geojson` so that each observed point can be paired with its reference for accuracy reporting.

### 1. Load the Data

1. Load the SWIR and VNIR files into QGIS and run the following checks:

- Do the panels in the SWIR and VNIR overlap?
- Make note of the CRS.

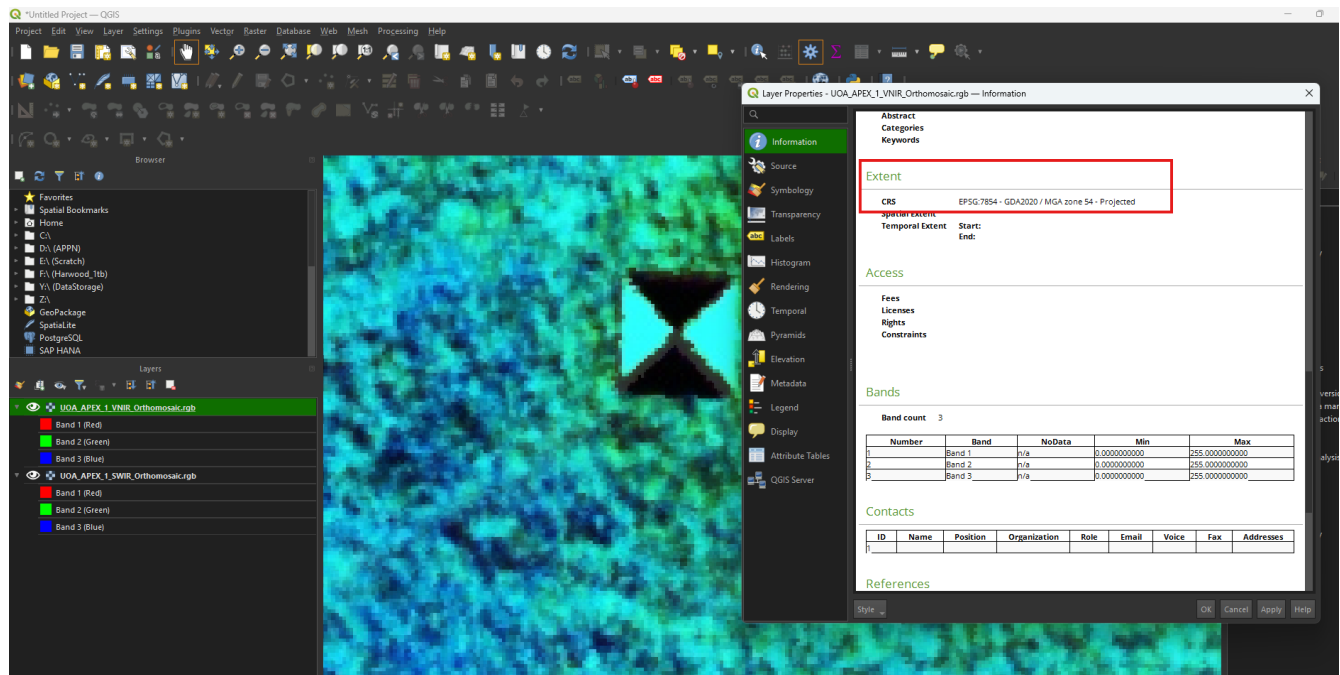


Figure: The VNIR is set to 50% opacity to confirm the panels overlap with the SWIR. The information panel is open on the SWIR to confirm the CRS (EPSG:7854 – GDA2020 / MGA zone 54). This CRS is for Roseworthy SA where this CALViS flight was collected.

### Caution

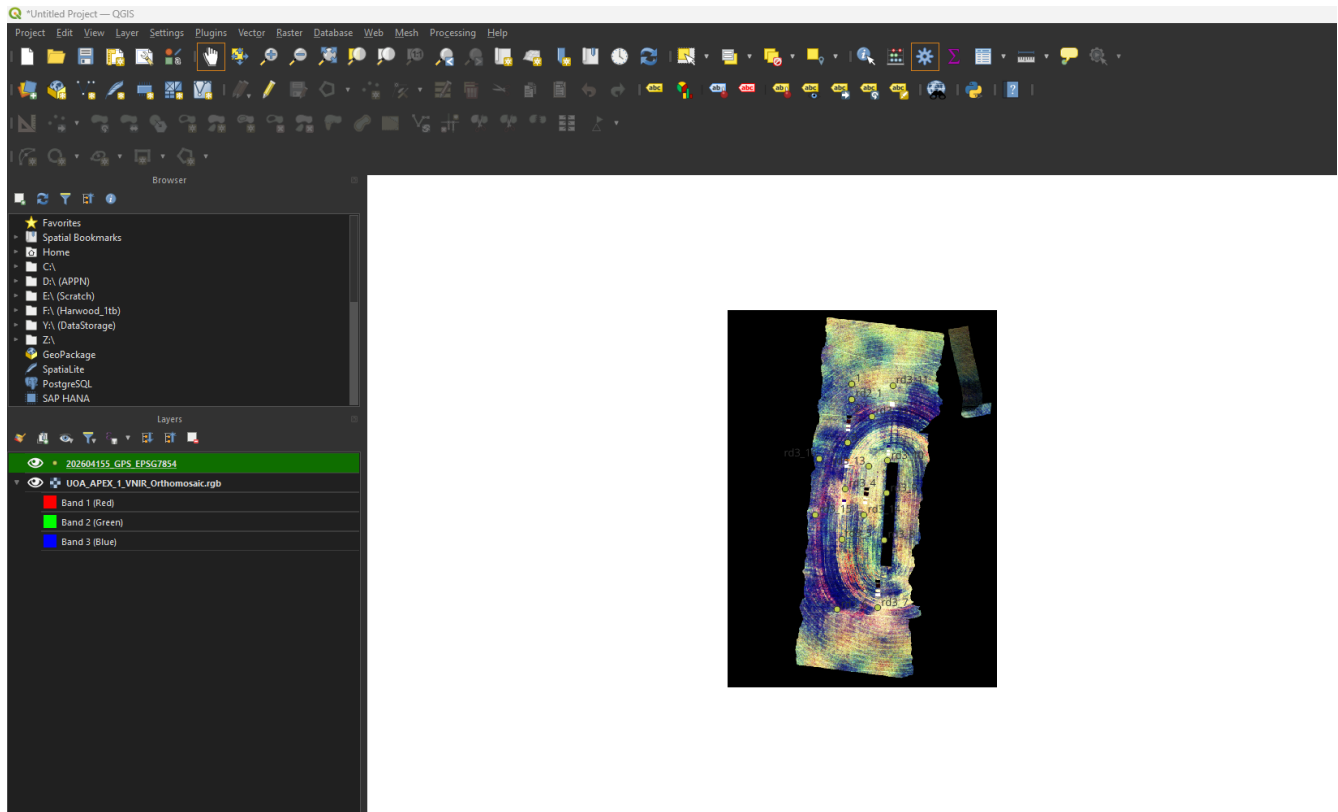
For CALViS, the SWIR and VNIR orthomosaics **must** be checked independently. The two sensors are georeferenced separately, and we have seen flights where the GNSS correction was applied cleanly to one orthomosaic

but was broken or offset in the other. Confirming each one on its own is the only reliable way to catch this.

Instead of transparency you can also toggle one image on and off to confirm the overlap is acceptable.

If overlap is fine:

2. Load the ground-truth GPS data (QC\_GCP\_groundtruth\_points.geojson) for your GCPs. The reference layer is loaded first so it is easy to name the digitised points on the drone imagery to match.



Example of the ground-truth points over a GCP visible in the CALViS orthomosaic:



## 2. Create the Vector Layer

1. Navigate to **Layer** → **Create Layer** → **New Shapefile Layer**.

Set the **File Name** to QC\_GCP\_points, the **Geometry type** to *Point*, and replace the pre-filled **id** field with GCP\_name of type **Text (string)**.

**New Shapefile Layer**

File name: QC\_GCP\_points.shp

File encoding: UTF-8

Geometry type: Point

Additional dimensions: ☒ None ☐ Z (+ M values) ☐ M values

Project CRS: EPSG:7854 - GDA2020 / MGA zone 54

**New Field**

Name:

Type: Text (string)

Length: 80 Precision:

Add to Fields List

**Fields List**

| Name     | Type   | Length | Precision |
|----------|--------|--------|-----------|
| GCP_name | String | 80     |           |

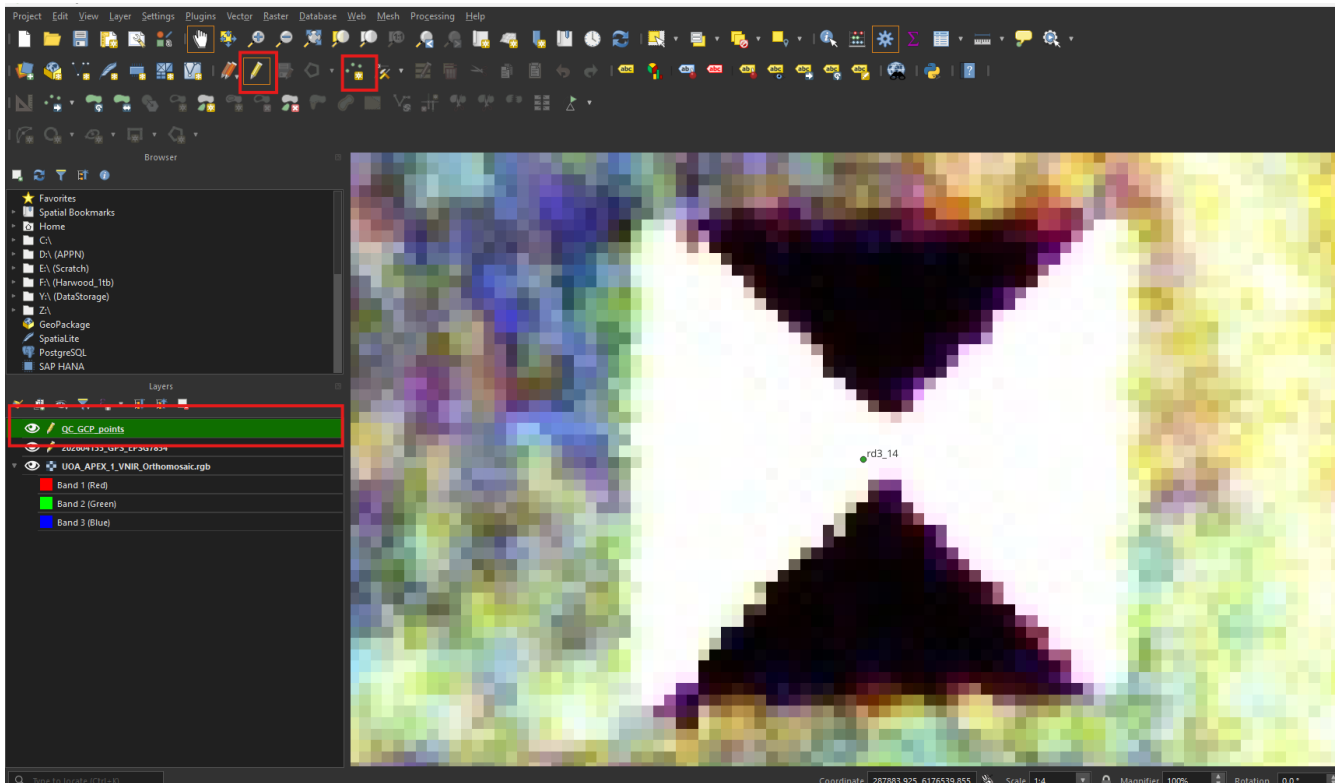
Remove Field

OK Cancel Help

- Set the **File Name** to QC\_GCP\_points.shp (it will be exported to GeoJSON in step 5).
- Set the **Geometry type** to *Point*.
- Set the **CRS** to match the orthomosaic.
- Add a single field — GCP\_name:
  - Select the pre-filled id field in the Fields list and click **Remove Field** (bottom right) to remove it.
  - Add GCP\_name as **Text (string)**.

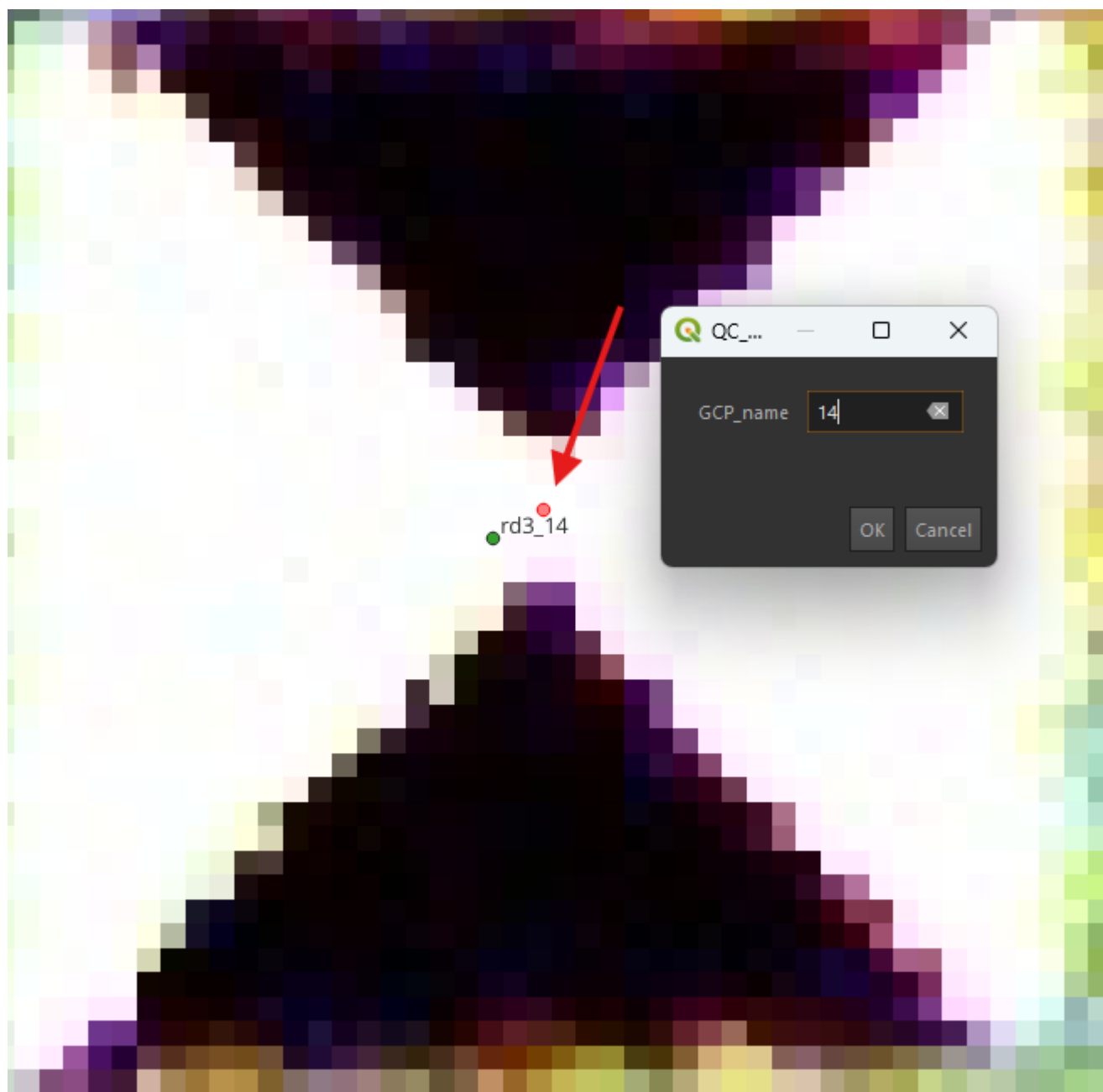
### 3. Annotating the GCPs To start annotating the drone orthomosaic:

1. Select QC\_GCP\_points in the **Layers** menu.
2. Click the pencil icon (**Toggle Editing**).
3. Click **Add Point Feature**.



4. Navigate to the centre of each GCP, left-click to drop a point, and enter the GCP\_name so it matches the corresponding ID in QC\_GCP\_groundtruth\_points.geojson. In the figure below the ground-truth point is green and the digitised (observed) point is red — the offset between them is the positional error being measured.





Repeat for every GCP visible in the orthomosaic.

## 5. Save the data as a GeoJSON

1. Right-click **QC\_GCP\_points** in the **Layers** menu → **Export** → **Save Features As...**
2. Set **Format** to *GeoJSON*, confirm the **File Name** and target folder (click the three dots to navigate), and double-check the **CRS** matches the orthomosaic.

Save Vector Layer as...

Format: GeoJSON

File name: D:\GPT\_LOCAL\20260415\run\_00\T1\_proc\QC\_data\QC\_GCP\_points.geojson

Layer name:

CRS: EPSG:7854 - GDA2020 / MGA zone 54

Encoding: UTF-8

☐ Save only selected features

▶ Select fields to export and their export options

☒ Persist layer metadata

▼ Geometry

Geometry type: Automatic

☐ Include z-dimension

☐ Force multi-type

▶ ☐ Extent (current: none)

▼ Layer Options

COORDINATE\_PRECISION: 15

RFC7946: NO

WRITE\_BBOX: NO

▶ Custom Options

☒ Add saved file to map

OK Cancel Help

## Accuracy reporting

Run the QA code at [https://github.com/ArdenB/APPN\\_GenricFileStorage](https://github.com/ArdenB/APPN_GenricFileStorage) (Code/DS02\_DatasetQA/) to generate a spatial accuracy report comparing the observed (drone) and reference (surveyed) GCP locations. The report produces per-point residuals and overall RMSE in X, Y, and Z.

### Important

**TODO:** Write this section and include lots of info.

## Spectral QC

Spectral QC is done by drawing polygons in GIS software over surfaces with known spectral values, then extracting and comparing them. This is done after all processing in software like GPT is complete.

CALViS and GOBI flights conducted prior to the *Operational Excellence in APPN Hyperspectral Imaging* SIF likely consist of one GRYFN reflectance panel (used to generate the ELM in the GRYFN Processing Tool) along with additional panels for validation.

File naming, format (GeoJSON preferred), and storage location for the polygons created below all follow the General QC Conventions above.

## Creating Geospatial Vector Data — GOBI & CALViS (QGIS)

The description below shows the procedure for creating GeoJSON files for the CALViS using GRYFN reflectance panels for ELM. The process for the GOBI is the same, but there is only the VNIR orthomosaic. The same workflow can be used to produce shapefiles if preferred — just choose *ESRI Shapefile* in place of *GeoJSON* in the format dropdown.

For the CALViS, in the products folder of the completed GPRO you will find the **.tif** files:

- X\_VNIR\_Orthomosaic.rgb
- X\_calvis\_sif\_cal\_20250925\_SWIR\_Orthomosaic.rgb

These are the RGB bands from the VNIR and SWIR files respectively. They can be easier to use than the full **.bin** files, though the procedure is the same.

### 1. Load the Data

1. Load both files into QGIS and run the following checks:

- Do the panels in the SWIR and VNIR overlap?
- Make note of the CRS.

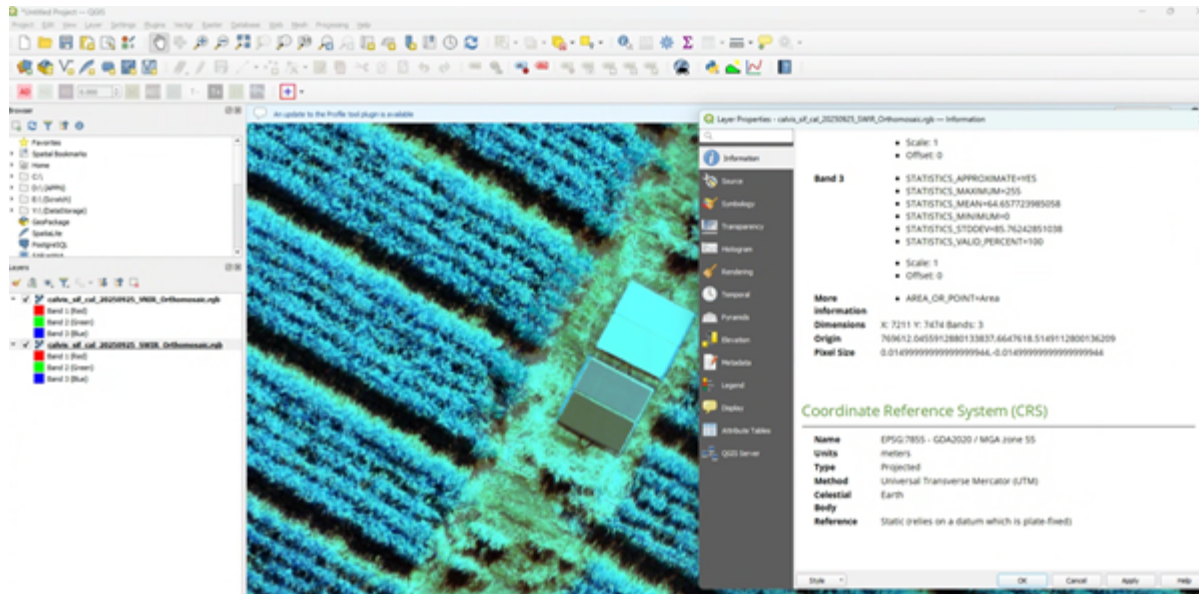
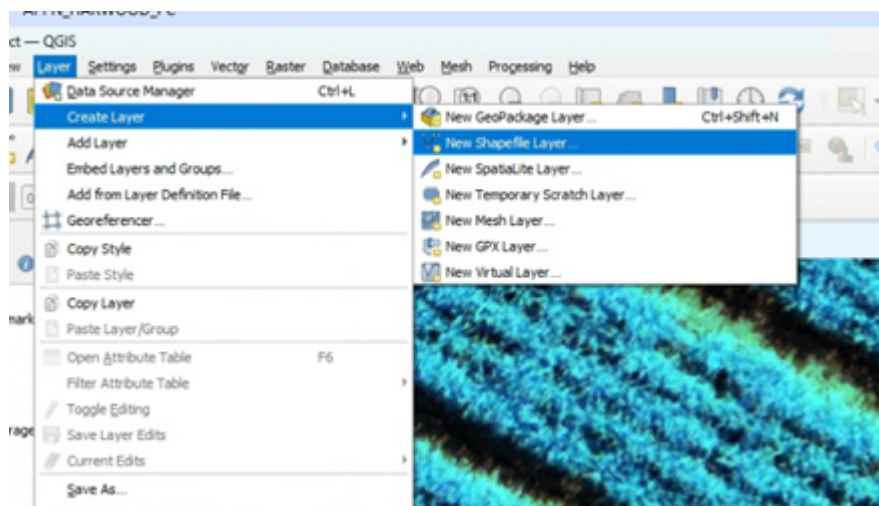


Figure: The VNIR is set to 50% opacity to confirm the panels overlap with the SWIR. The information panel is open on the SWIR to confirm the CRS (EPSG:7855 – GDA2020 / MGA zone 55). This CRS is for Narrabri NSW where this CALViS flight was collected.

### 2. Create the Vector Layer **Important**

**TODO:** Richard please check and update this section.

1. Navigate to **Layer → Create Layer → New GeoPackage Layer...** for GeoJSON output, or **New Shapefile Layer...** if producing a shapefile. Either way you will set the output **File Name** with the appropriate extension (**.geojson** preferred, **.shp** accepted).



### Tip

To save directly as GeoJSON you can also create a temporary scratch layer and then **Export** → **Save Features As...** → **GeoJSON** in step 5.

2. Set the **File Name** to `QC_ELM_Panels.geojson` (see the naming conventions table for the correct name given your use case).
3. Set the **Geometry type** to *Polygon*.
4. Set the **CRS** to match your dataset.
5. Add a single field — `Panel_ref`:
  - Select the pre-filled `id` field in the Fields list and click **Remove Field** (bottom right) to remove it.
  - Set `Panel_ref` as an **Integer** between 0–100. It is the percentage reflectance (e.g. 11, 30, 56, 82 for GRYFN panels).

### Important

You need to click **Add to Fields List** once you populate "New Field".

**New Shapefile Layer**

File name: QC\_ELM\_Panels.shp

File encoding: UTF-8

Geometry type: Polygon

Additional dimensions: None (selected), Z (+ M values), M values

Project CRS: EPSG:7855 - GDA2020 / MGA zone 55

**New Field**

Name: Panel\_ref

Type: Integer (32 bit)

Length: 10 Precision: 0

Add to Fields List

**Fields List**

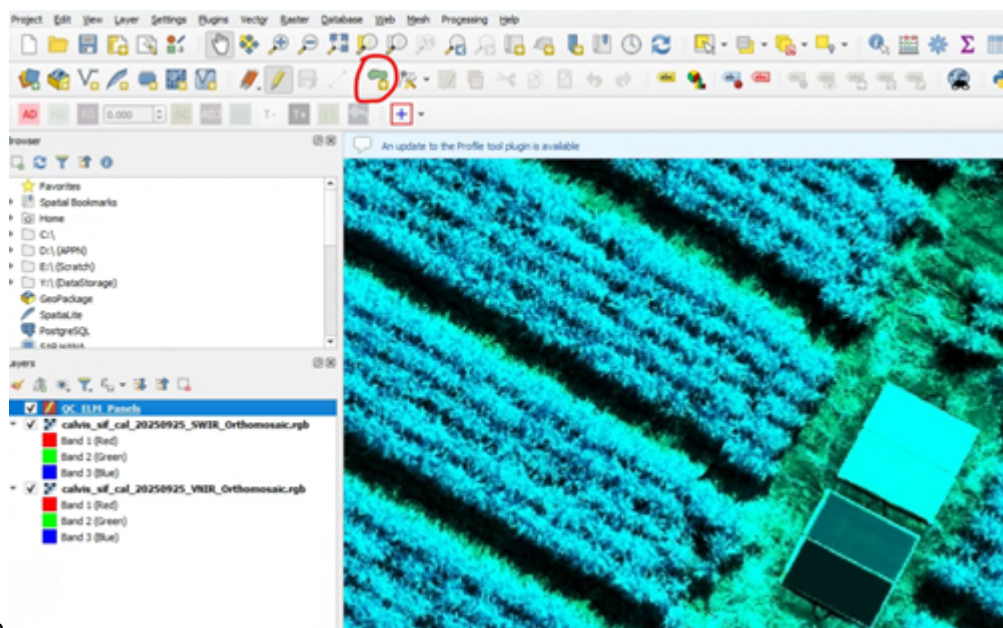
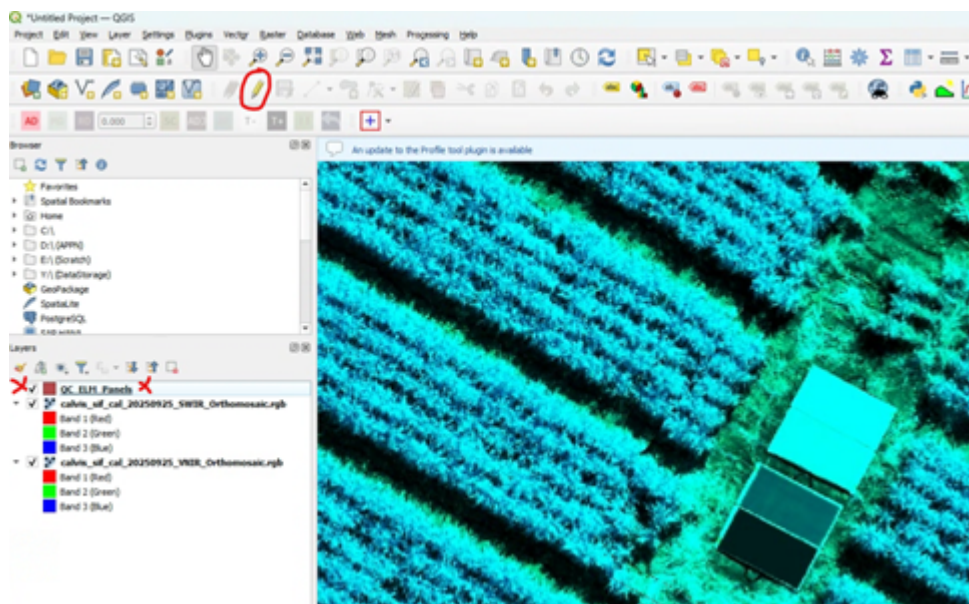
| Name      | Type    | Length | Precision |
|-----------|---------|--------|-----------|
| Panel_ref | Integer | 10     |           |

Remove Field

OK Cancel Help

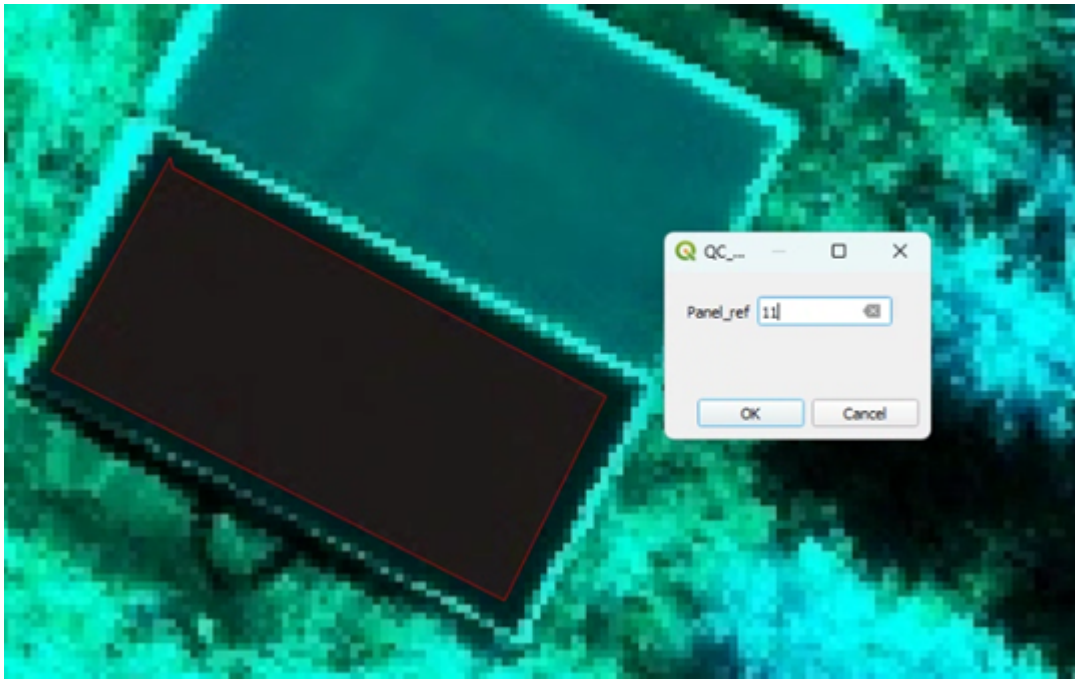
**3. Draw Boxes over the Panels** The boxes should be polygons drawn inside the panel. The points should be drawn to cover as much of the panel as possible without including edge effects (2–3 pixels from the edge of the panel). If there are gaps or missing values within the panel, they should be included.

1. Select **QC\_ELM\_Panels** in the **Layers** menu.
2. Click the pencil icon (**Toggle Editing**).



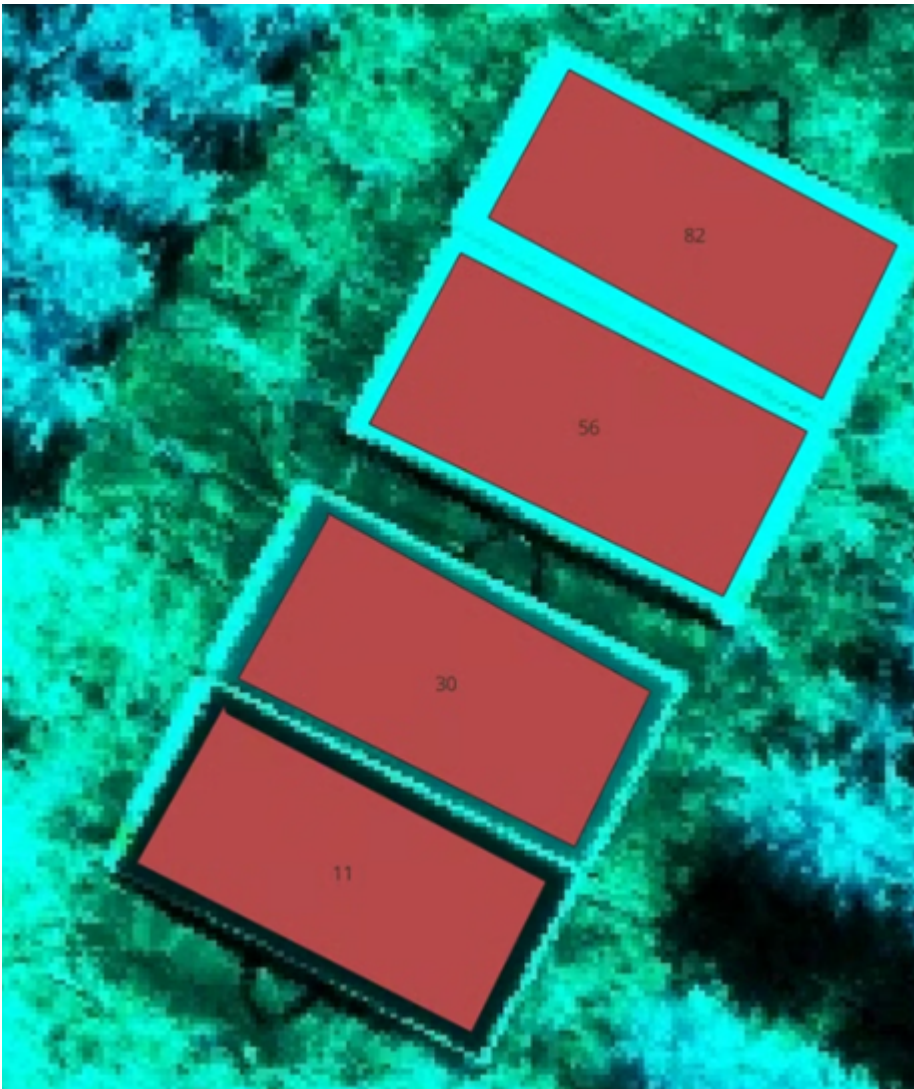
3. Click **Add Polygon Feature**
4. Click four points around a specific reflectance panel, then right-click to close the polygon. Repeat for all panels (e.g. 11, 30, 56, 82).





#### 4. Sanity Check Labels

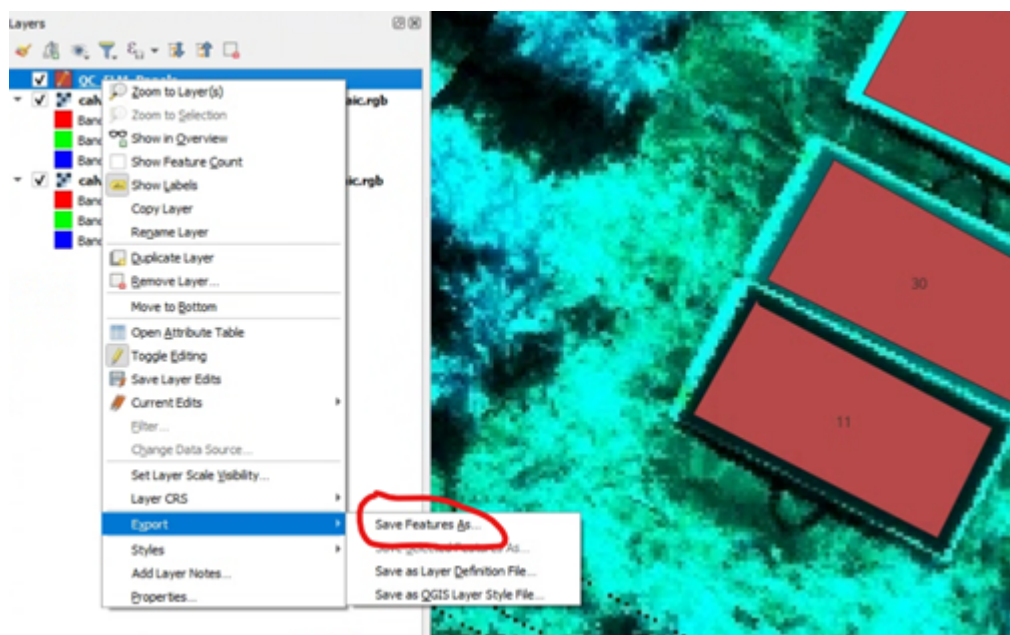
1. Double-click QC\_ELM\_Panels in the **Layers** menu.
2. In **Layer Properties**, click **Labels** and select **Single Labels**.



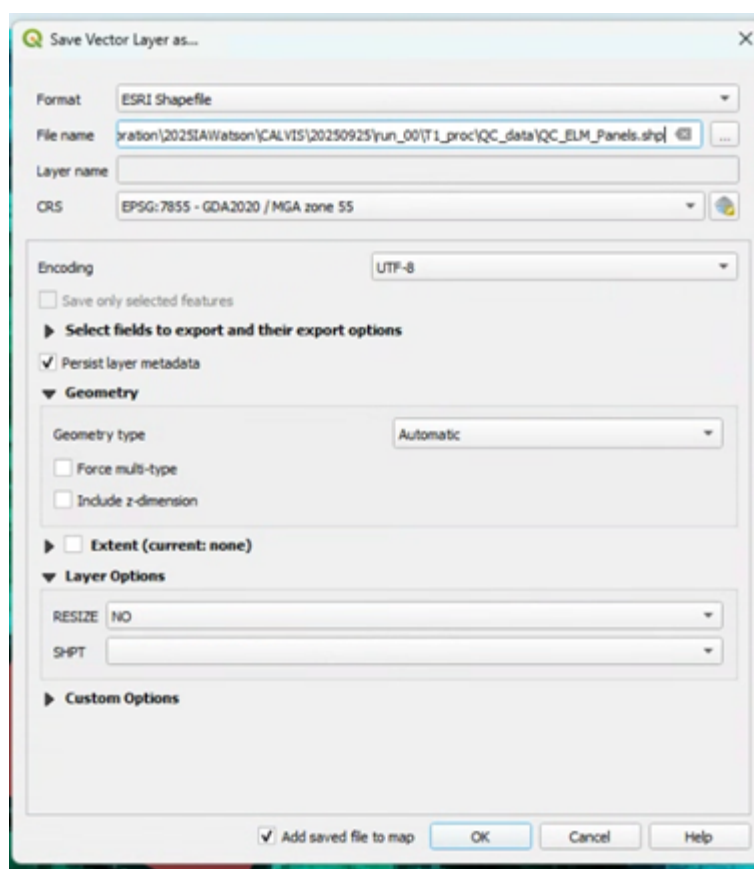
## 5. Save the File

1. Right-click QC\_ELM\_Panels in the **Layers** menu → **Export** → **Save Features As**.





2. Set the **Format** to *GeoJSON* (preferred) — or *ESRI Shapefile* if required. Make sure the **File Name** is correct and in the right folder (click the three dots to navigate). Double-check the **CRS**.



## Extracting Pixels into a Table

Arden Burrell has made a Python script that can go through the APPN standard folder structure, extract the values into a table, and save that as a `.csv` or `.parquet` file automatically. The code is available from:

[https://github.com/ArdenB/APPN\\_GenricFileStorage](https://github.com/ArdenB/APPN_GenricFileStorage)

- **Script:** Code/DS02\_DatasetQA/QA00\_ELMvalidation.py
- **README:** Code/DS02\_DatasetQA/README.md

If nodes choose to extract the points using other means, the tables should have the following columns:

band, wavelength, value, Panel\_ref, node, project, site, sensor, date, run, panel\_name, type, gpro\_nu

---

## LiDAR QC

### Warning

This section is still a work in progress.